RAD Sem01_2627 · PART 1 PROJECT PROPOSAL

SaigonBite: one platform to spread lunch-hour orders across hubs

Online Food Delivery Hub Platform (OFDH)

Nguyen Thanh Dat · 722I0004
UC-01 Place an order, UC-03 Manage hubs

Le Minh Quan · 523K0049
UC-02 Deliver an order, UC-04 Reports

Quan · ⟦ACTUAL m3_id: student ID⟧
UC-05 Confirm or reject, UC-06 Menus

Team02 · Presented live on 26 October 2026

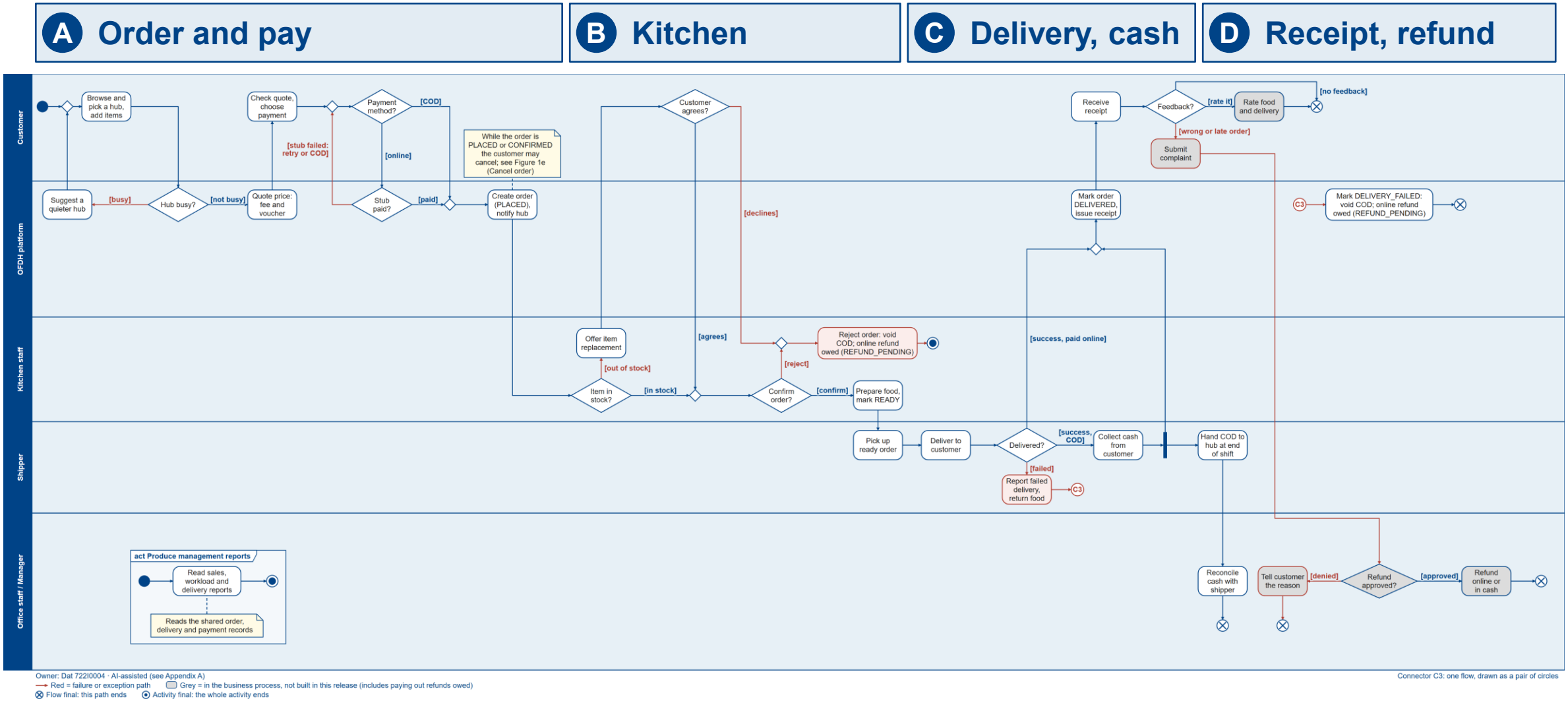
Paper tickets hide the queue: one hub overloads, another idles

Five pains from the case, and what the platform does about each.

Pain (from the case)	What the platform does
One hub overloaded, another idle	Busy-hub block, quieter-hub suggestion
Cash handed over late, with mistakes	Shipper records cash per order
Ingredient runs out; staff phone customers	Kitchen rejects, or replaces with consent
Tourists cannot register or pay	Local and tourist types; COD for all
No reliable reports	Reports from shared records

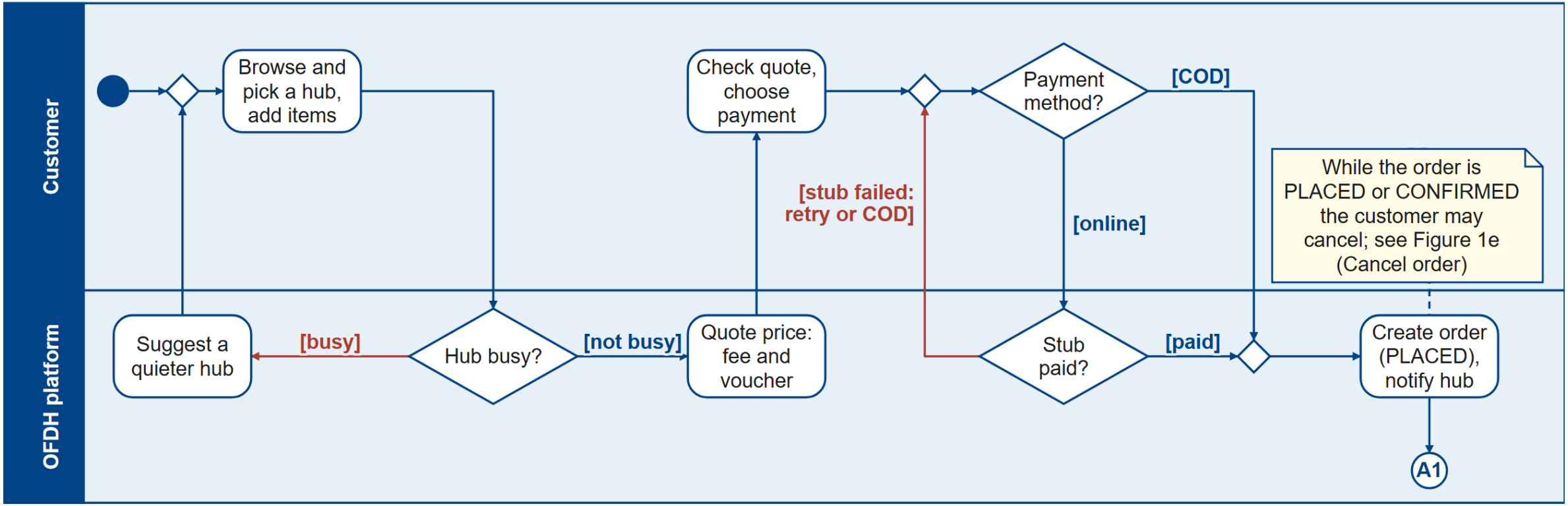
Vision One order record that customer, kitchen, shipper and manager all read.

One lunch order crosses five roles



A busy hub redirects the customer before any order exists

Part A: Order, quote and payment



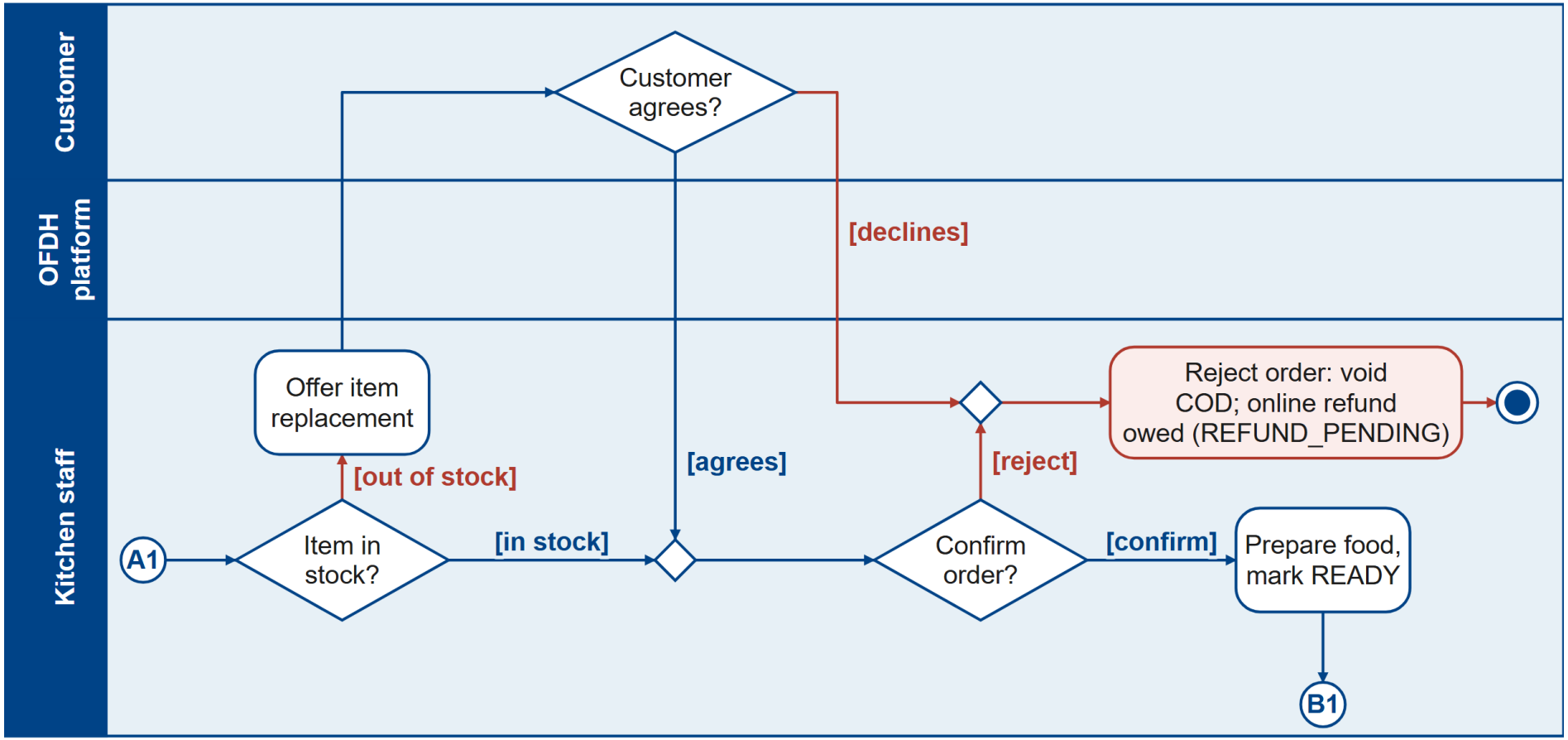
Owner: Dat 722I0004 · AI-assisted (see Appendix A)

Connectors: A1 continues in Part B

→ Red = failure or exception path

The kitchen confirms, offers a replacement, or rejects

Part B: Kitchen and item replacement



Owner: Dat 722I0004 · AI-assisted (see Appendix A)

Connectors: A1 comes from Part A; B1 continues in Part C

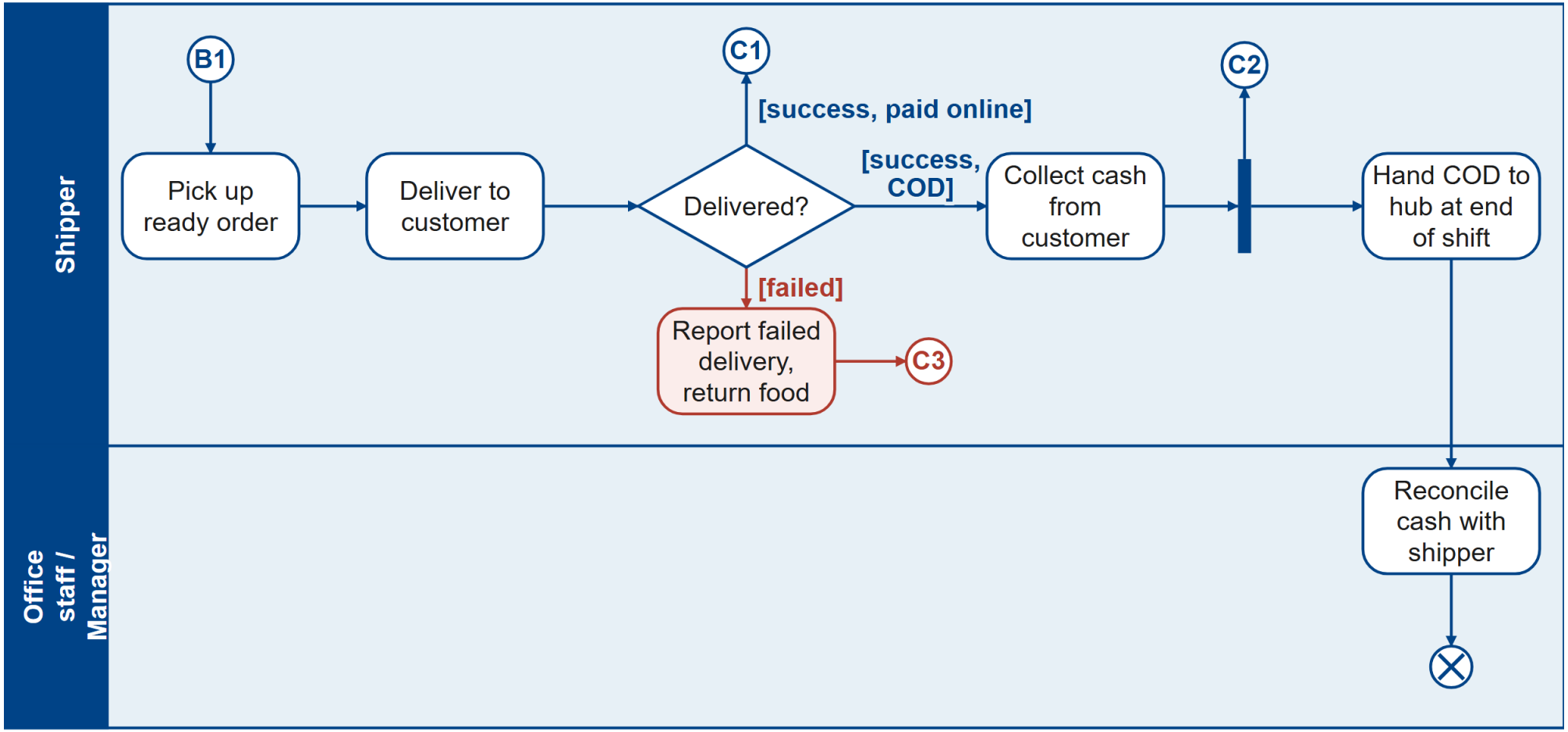
→ Red = failure or exception path

◉ Activity final: the whole activity ends

Part E The customer may cancel until cooking starts: a separate activity (proposal Figure 1e).

Delivered is not paid: cash is collected, then reconciled

Part C: Delivery and cash



Owner: Dat 722I0004 · AI-assisted (see Appendix A)

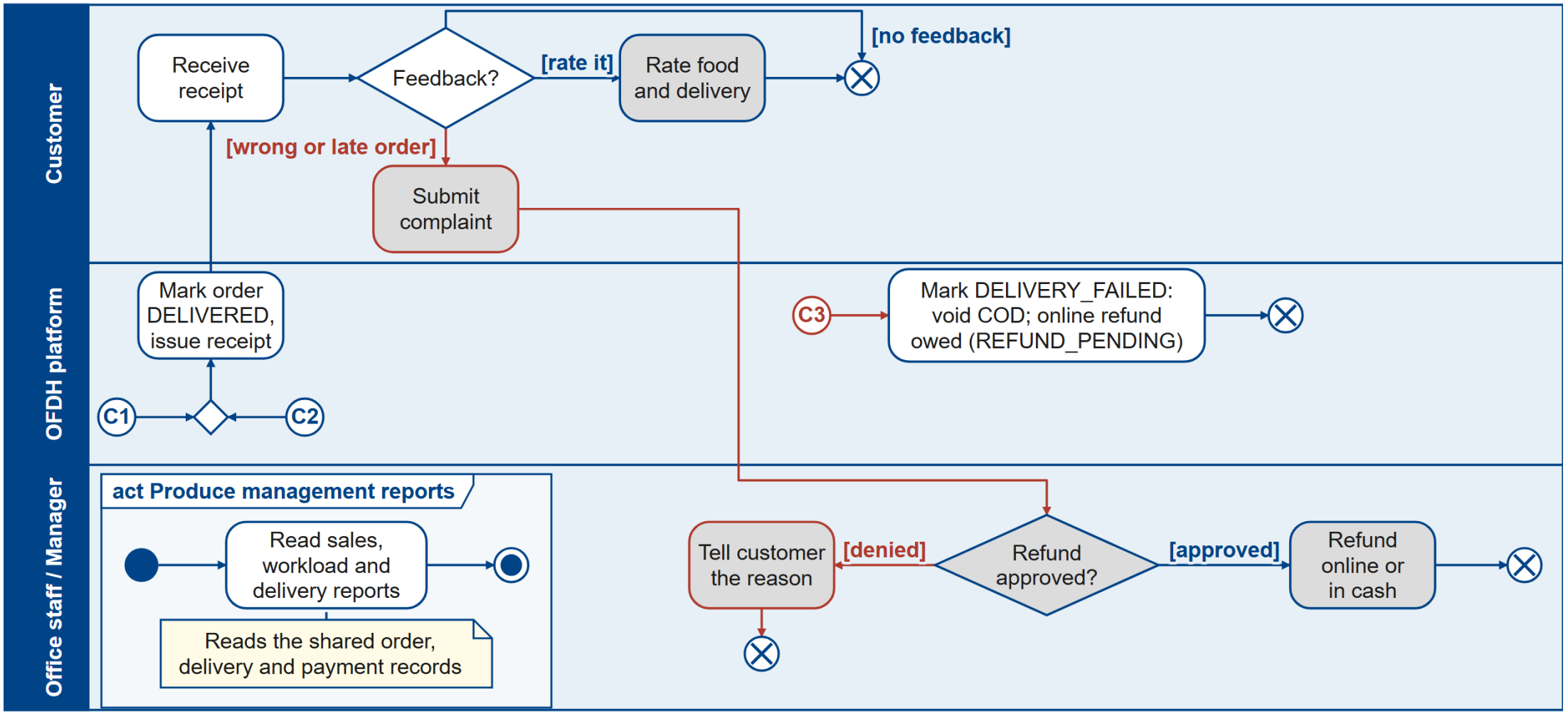
→ Red = failure or exception path

⊗ Flow final: this path ends

Connectors: B1 comes from Part B; C1, C2, C3 continue in Part D

A failure means a full refund owed; only a complaint needs a decision

Part D: Receipt, failed delivery, refund and reports



Owner: Dat 722I0004 · AI-assisted (see Appendix A)

Connectors: C1, C2, C3 come from Part C

→ Red = failure or exception path Grey = in the business process, not built in this release (includes paying out refunds owed)

⊗ Flow final: this path ends ◉ Activity final: the whole activity ends

Actors and responsibilities

Six roles in the business and one outside system.

Actor	Responsibility	Main use case
Customer	Orders, pays, may cancel or complain	UC-01
Kitchen staff	Confirms, rejects or replaces; cooks	UC-05
Shipper	Delivers, records cash per order	UC-02
Office staff	Reconciles cash, decides complaints	in process
Admin	Keeps hubs, capacity and menus	UC-03, UC-06
Manager	Reads sales, workload, delivery	UC-04
Payment provider	Online result (our stub)	UC-01

Six main use cases, two per student, cover all four groups

ID	Use case	Actor	Group	Owner	State machine
UC-01	Place an order (#3)	Customer	(a) ordering	Dat	Order
UC-02	Deliver an order (#10)	Shipper	(b) fulfilment	Quan L.	Delivery
UC-03	Manage hubs, capacity (#13)	Admin	(c) full CRUD	Dat	Hub
UC-04	Produce reports (#16)	Manager	(d) report	Quan L.	none (read-only)
UC-05	Confirm or reject an order (#7)	Kitchen staff	(b) fulfilment	Quan	uses Order
UC-06	Manage brands, menu items (#14)	Admin	(c) full CRUD	Quan	MenuItem

Built today (Iteration 1) UC-01 subset: one persisted COD order

Planned build for Part 2 UC-01 to UC-06 in one application

Analysed only Cancel, voucher admin, accounts, order tracking

Deferred Complaint, refund payout, rating

Short iterations, because the client's rules are still open

Six iterations; each one analyses, designs, builds, tests and ends with a team review.

Why iterative

1 Answers pending
Client figures arrive later

2 One shared Order
Six use cases grow one record

3 Fixed deadlines
Three presentations; team of three

4 Riskiest first
Capacity in I2, payment in I3

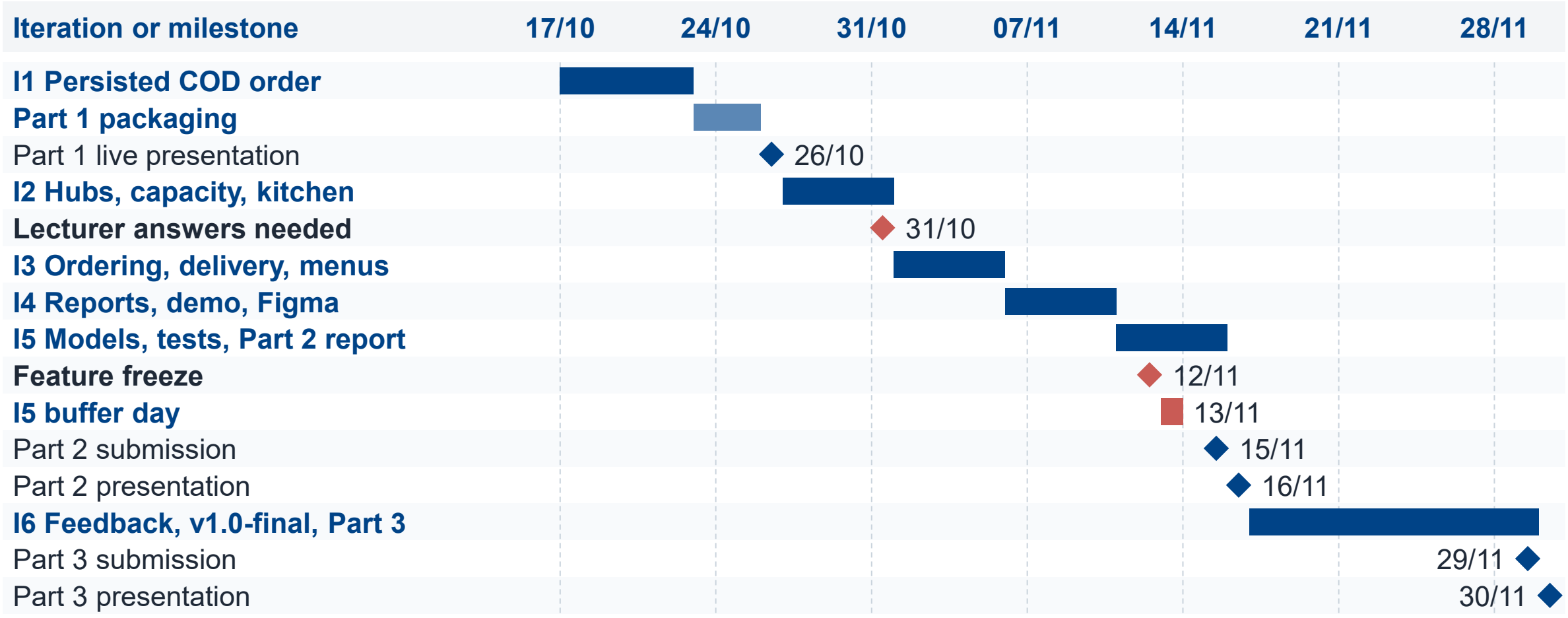
Stack and three layers (team choice)

View	Flask + Jinja2 templates
Controller	One per use case
Domain	Order, Hub, Payment, MenuItem
DAO	sqlite3, hand-written SQL
Database	SQLite, one database

Python 3.13 · pytest · Flask-Login
Lecturer asked to confirm Python

Six iterations; Part 2 never depends on the last one

Weekly view; red marks are decision points.



Contingency decide scope by 31/10 · freeze 12/11 · buffer day 13/11 · I6 only for feedback

Iteration 1 runs the six core processes in six days

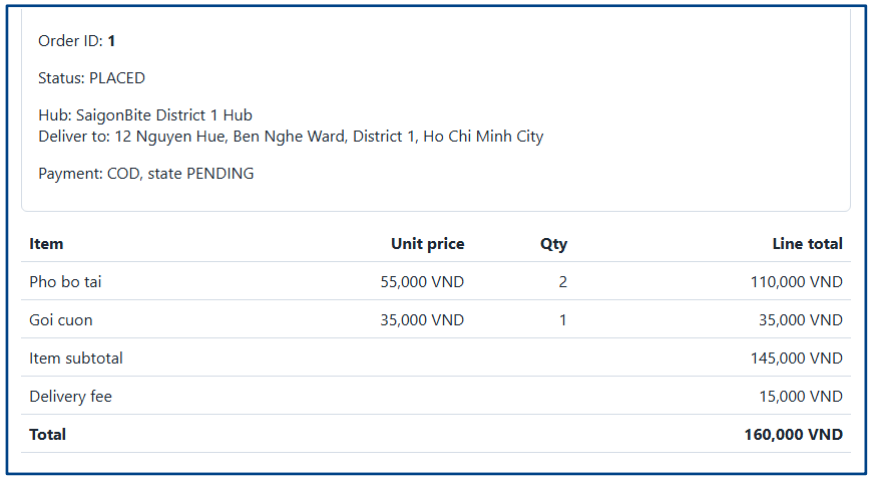
17–22/10, two people, about 4 focused hours each per day (48 person-hours).

Day	Core process	Dat (UC-01)	Quan L. (foundation)	Evidence
D1	**CP1–2 Identify, plan**	Vision, backlog, UC-01 stories	Repo, skeleton, events, actors	Skeleton runs; backlog v1
D2	**CP3 Discover details**	UC-01 description, process diagram	Domain model, CRUD inventory	Names fixed
D3	**CP3–4 Design components**	Activity, SSD, Order states	Schema, seed, UC-02 draft	SSD and schema agree
D4	**CP4–5 Build, test**	Controller, Order code	DAOs, OrderDAO insert	COD order saved
D5	**CP5 Integrate**	Checkout page, pytest	Role checks, README	pytest run recorded
D6	**CP6 Test, deploy**	Fix defects, acceptance run	Second-machine run	Result; I1 review

Iteration 1: a COD order saved end to end; 9 of 9 criteria met

As of 20/10, from the repository and the pytest output.

Measure	Plan	Actual
Criteria met	9 of 9	9 of 9
Tests	AC-4 to AC-8	93 passed, 0 failed
Second machine	Passes	Mac (Docker): app and tests pass
Code review	On D6	20/10: 3 defects found, all fixed
Person-hours	48	Not logged per day
Roadmap change	None	Third member from I2



Order ID: **1**

Status: PLACED

Hub: SaigonBite District 1 Hub
Deliver to: 12 Nguyen Hue, Ben Nghe Ward, District 1, Ho Chi Minh City

Payment: COD, state PENDING

Item	Unit price	Qty	Line total
Pho bo tai	55,000 VND	2	110,000 VND
Goi cuon	35,000 VND	1	35,000 VND
Item subtotal			145,000 VND
Delivery fee			15,000 VND
Total			**160,000 VND**

Saved order: ID 1, PLACED, COD, total 160,000 VND

Status Built early on 19/10 with AI assistance; reviewed and fixed by the team on 20/10/2026.

Lesson Green tests are not proof: review found a bug 81 tests missed. Every finding now gets a test.

Unconfirmed rules and the capacity race are the biggest risks

Each risk has a warning sign we watch and one owner.

ID	Risk	L / I	Owner	Mitigation
R-01	**CRUD coverage rejected**	Med / High	Dat	Decide by 31/10; reuse UC-06 admin pattern
R-02	**Rules unconfirmed or changed**	High / Med	Dat	Assumption Log; values in configuration
R-03	**Peak-hour capacity race**	Med / High	Dat	One BEGIN IMMEDIATE transaction; boundary test
R-04	**Integration drift between use cases**	Med / High	Dat	Shared schema, seed; demo at I3, I4
R-05	**Member absent; new member ramping up**	Med / High	Dat	Pairing in I2; buffer day 13/11

What we need from you

We keep building on our defaults until the answers arrive.

Client questions (with our default)

Q1 Workload Busy and quiet hub? Default: at capacity; quiet gets 5,000 VND off.

Q2 Delivery Distance, fee? Default: base + per km.

Q3 Cancel Until when? Default: before cooking; refund in full.

Q4 Payment Online options? Default: wallet, card.

Q5 Reports Which date? Default: delivered date.

Questions for the lecturer

L1 Analysis plus seed data enough for CRUD?

L2 Team-built payment stub acceptable?

L3 Defer complaint, refund, rating?

L4 Python with Flask acceptable?

L5 Third member, UC-05 and UC-06 accepted?

Close Please answer by 31/10. Iteration 2 starts on 27/10.